REACT NATIVE NOTES

1. What is React Native? 👋

React Native is an open-source framework used to develop mobile applications using JavaScript, TypeScript, and React.

React Native is developed and maintained by Meta and the open-source community.

Using React Native, we can create applications for:

- Android
- iOS

Simple Definition:

React Native is a framework that allows us to build mobile applications using React and JavaScript.

Real-Life Example

Imagine you want to develop a food delivery application.

Your application may contain:

- Home Screen
- Restaurant List
- Food Details Screen
- Cart Screen
- Profile Screen

Using React Native, we can create these screens for both Android and iOS.

We do not need to develop two completely separate applications.

React JS vs React Native

React JS:

- Used mainly for web application development.
- Uses HTML elements such as div, h1, p, and button.
- Runs in web browsers.

React Native:

- Used for mobile application development.
- Uses components such as View, Text, and TextInput.
- Runs as a mobile application.

Example:

React JS:

```
<div>
  <h1>Hello Students</h1>
</div>
```

React Native:

```
<View>
  <Text>Hello Students</Text>
</View>
```

Remember:

React JS is mainly used to build web interfaces.

React Native is used to build mobile application interfaces.

---

2. Why Do We Use React Native?

Normally, Android and iOS applications can be developed using different technologies.

For example:

Android Development:

- Kotlin
- Java

iOS Development:

- Swift

If we develop applications separately, we may need different codebases for Android and iOS.

React Native helps us solve this problem.

Main Reason

Write code once and reuse much of it for Android and iOS.

This approach is called cross-platform development.

Real-Life Example

Suppose a company wants to develop an e-commerce mobile application.

The application contains:

1. Login Screen
2. Home Screen
3. Products Screen
4. Cart Screen
5. Orders Screen

Using React Native, developers can share most of the application code between Android and iOS.

Some platform-specific code may still be required.

---

3. Advantages of React Native

1. Cross-Platform Development

We can develop Android and iOS applications using a shared codebase.

2. JavaScript and TypeScript Support

We can use JavaScript or TypeScript to develop mobile applications.

3. React Knowledge Is Reusable

If you already know React JS, you can reuse concepts such as:

- Components
- Props
- State
- Hooks
- Context API

4. Fast Refresh

When we modify our application code, React Native can quickly update the application during development.

We usually do not need to restart the application for every small change.

5. Reusable Components

We can create a component once and use it in multiple screens.

For example, a custom button component can be used on Login, Register, and Profile screens.

6. Native User Interface

React Native uses native platform views for many of its UI components.

This helps applications provide a mobile-friendly experience.

7. Large Community

React Native has a large developer community and many useful libraries.

8. Backend Integration

React Native applications can communicate with backend technologies such as:

- Node.js
- Express.js
- Python FastAPI

They can also use APIs to work with databases such as MongoDB and PostgreSQL.

---

4. What is Expo?

Expo is a framework and a collection of tools built around React Native.

It makes React Native application development easier.

Simple Definition:

Expo helps us create, run, test, and build React Native applications with less initial configuration.

Why Do We Use Expo?

Expo provides tools for:

- Creating a React Native project
- Running a development server

- Testing applications on mobile devices
- Using common device features
- Building Android and iOS applications

What is Expo Go?

Expo Go is a mobile application used to run compatible Expo projects during development.

For example:

You write React Native code on your laptop.

You start the Expo development server.

You scan the QR code using Expo Go.

Your application opens on your mobile device.

Important: Expo Go is mainly useful for learning and quick testing. Applications requiring custom native modules may need a development build.

---

5. How to Create a React Native Application Using Expo

We will create our first React Native application step by step.

Step 1: Install Node.js

First, install Node.js on your computer.

Website:

https://nodejs.org

After installation, open the terminal.

Check the Node.js version:

```
node -v
```

Check the npm version:

```
npm -v
```

If both commands display version numbers, Node.js and npm are installed.

Step 2: Create a React Native Application

Open your terminal.

Run:

```
npx create-expo-app@latest MyFirstApp
```

This command creates a new Expo application.

Explanation:

- npx: Runs a package command.
- create-expo-app: Creates an Expo project.
- MyFirstApp: Our application name.

The default template may vary with the Expo SDK version.

Step 3: Open the Project

Move inside the project folder:

```
cd MyFirstApp
```

Step 4: Start the Development Server

Run:

```
npx expo start
```

This command starts the Expo development server.

You will see a QR code in the terminal.

Step 5: Install Expo Go

Install Expo Go on your Android or iOS mobile device.

For a beginner, an Android device with Expo Go is a convenient option.

Step 6: Run the Application

1. Connect your laptop and mobile device to the same Wi-Fi network.
2. Open Expo Go on your mobile device.

3. Scan the QR code displayed in the terminal.
4. Wait for the application to load.

If the local network connection does not work, try:

```
npx expo start --tunnel
```

Your first React Native application is now ready for development.

Step 7: Understand the Project Structure

A project created using Expo Router may contain:

```
MyFirstApp
│
├── app
│   ├── _layout.tsx
│   └── index.tsx
│
├── assets
├── components
├── package.json
└── tsconfig.json
```

app/

Contains application screens and routes.

index.tsx

Usually represents the initial screen for its route.

_layout.tsx

Defines the layout and navigation structure for routes inside a folder.

assets/

Contains images, fonts, and other static resources.

components/

Contains reusable components.

package.json

Contains project dependencies and scripts.

Note: Folder names and files may differ depending on the selected Expo template.

---

6. Core Components in React Native

React Native provides built-in components to create mobile application interfaces.

We will learn:

1. View
2. Text
3. Button
4. TextInput
5. Pressable
6. Alert

6.1 View Component

View is a core component used to create layouts and group other components.

It is similar in purpose to a div element in React JS.

Syntax:

```
<View>
  {/* Other components */}
</View>
```

Example

```
import { View, Text } from "react-native";

export default function Home() {
  return (
    <View>
      <Text>Welcome to React Native</Text>
      <Text>My First Mobile App</Text>
    </View>
  );
}
```

Explanation:

1. We import View and Text from React Native.
2. View works as a container.

3. Text displays text on the mobile screen.
4. Both Text components are placed inside View.

Remember:

View is mainly used for creating layouts.

6.2 Text Component

Text is used to display text inside a React Native application.

In React JS, we use elements such as h1, p, and span.

In React Native, we commonly use Text.

Example

```
import { View, Text } from "react-native";

export default function Home() {
  return (
    <View>
      <Text>Hello Students!</Text>
      <Text>Welcome to VK AI TECH</Text>
    </View>
  );
}
```

Explanation:

- The first Text displays "Hello Students!"
- The second Text displays "Welcome to VK AI TECH"

6.3 Button Component

Button is used to display a clickable button.

React Native provides a built-in Button component.

Example

```
import { View, Button } from "react-native";
```

```
export default function Home() {
  return (
    <View>
      <Button
        title="Click Me"
        onPress={() => console.log("Clicked")}
      />
    </View>
  );
}
```

Explanation:

- Button creates a clickable button.
- title defines the button text.
- onPress runs a function when the user presses the button.

Remember:

In React JS, we commonly use onClick.

In React Native, we commonly use onPress.

6.4 TextInput Component

TextInput is used to accept input from the user.

For example:

- Name
- Email
- Password
- Search Text

Example

```
import { useState } from "react";

import {
  View,
  Text,
  TextInput
} from "react-native";

export default function Home() {
```

```
  const [name, setName] = useState("");

  return (
    <View>
      <Text>Enter Your Name</Text>

      <TextInput
        placeholder="Enter name"
        value={name}
        onChangeText={setName}
      />

      <Text>Hello {name}</Text>
    </View>
  );
}
```

Explanation:

1. useState creates a state variable called name.
2. TextInput allows the user to type.
3. value connects TextInput with state.
4. onChangeText updates the state.
5. The Text component displays the entered name.

Important TextInput Props

placeholder

Displays hint text when the input is empty.

value

Stores the current input value.

onChangeText

Runs when the user changes the text.

keyboardType

Defines the type of keyboard.

Example:

```
<TextInput
  placeholder="Enter Email"
  keyboardType="email-address"
```

```
/>
```

secureTextEntry

Hides entered text, commonly used for passwords.

Example:

```
<TextInput
  placeholder="Enter Password"
  secureTextEntry
/>
```

---

7. What is onPress?

onPress is an event handler prop.

It is used to perform an action when the user presses a component that supports press events.

Example

```
<Button
  title="Login"
  onPress={() => console.log("Login Pressed")}
/>
```

When the user presses the Login button, the function executes.

Example Using a Separate Function

```
import { Button } from "react-native";

export default function Home() {
  const handleLogin = () => {
    console.log("Login Successful");
  };

  return (
    <Button
      title="Login"
      onPress={handleLogin}
    />
  );
```

```
}
```

Remember:

Pass the function reference when you want it to run only after pressing.

Use:

```
onPress={handleLogin}
```

Do not use:

```
onPress={handleLogin()}
```

The second approach calls the function immediately during rendering.

---

8. What is Alert?

Alert is a React Native API used to display native alert dialogs.

It is commonly used to show:

- Information messages
- Warning messages
- Confirmation dialogs

Example

```
import {
  View,
  Button,
  Alert
} from "react-native";

export default function Home() {
  const showMessage = () => {
    Alert.alert(
      "Welcome",
      "Hello Students!"
    );
  };

  return (
```

```
    <View>
      <Button
        title="Show Alert"
        onPress={showMessage}
      />
    </View>
  );
}
```

Explanation:

1. We import Alert from React Native.
2. We create a function called showMessage.
3. Alert.alert() displays a native alert.
4. The function runs when the button is pressed.

Note: Alert.alert is primarily a native Android and iOS API. Web support differs.

---

9. Styling in React Native

Styling means changing the appearance of components.

For example:

- Background color
- Text color
- Font size
- Padding
- Margin
- Width
- Height
- Alignment

React Native styles are written using JavaScript objects.

There are two common approaches:

1. Inline Styling
2. StyleSheet API

9.1 Inline Styling

Inline styling means writing styles directly inside the component.

Example

```
import { View, Text } from "react-native";

export default function Home() {
  return (
    <View
      style={{
        flex: 1,
        backgroundColor: "black",
        justifyContent: "center",
        alignItems: "center"
      }}
    >
      <Text
        style={{
          color: "white",
          fontSize: 24,
          fontWeight: "bold"
        }}
      >
        Hello React Native
      </Text>
    </View>
  );
}
```

Explanation

flex: 1

Allows the View to fill the available space in its parent.

backgroundColor

Sets the background color.

justifyContent: "center"

Centers children along the main flex direction.

alignItems: "center"

Centers children along the cross axis.

color

Changes the text color.

fontSize

Changes the text size.

fontWeight

Changes the thickness of the text.

Important Point

React Native style properties use camelCase.

For example:

CSS:

background-color: black;
font-size: 24px;

React Native:

```
{
  backgroundColor: "black",
  fontSize: 24
}
```

9.2 StyleSheet API

StyleSheet is an API provided by React Native.

It helps us organize styles separately from JSX.

Example

```
import {
  View,
  Text,
  StyleSheet
} from "react-native";

export default function Home() {
  return (
    <View style={styles.container}>
      <Text style={styles.heading}>
        Welcome Students
```

```
      </Text>

      <Text style={styles.description}>
        Learning React Native
      </Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: "#ffffff",
    justifyContent: "center",
    alignItems: "center"
  },

  heading: {
    fontSize: 26,
    fontWeight: "bold",
    color: "blue"
  },

  description: {
    fontSize: 18,
    marginTop: 10,
    color: "gray"
  }
});
```

Explanation

Step 1:

Import StyleSheet from React Native.

Step 2:

Create styles using StyleSheet.create().

Step 3:

Define style names such as container and heading.

Step 4:

Apply styles using the style prop.

Example:

style={styles.container}

Inline Styling vs StyleSheet

Inline Styling:

- Styles are written directly inside JSX.
- Useful for small or dynamic styles.
- Can make large components harder to read.

StyleSheet:

- Styles are defined separately.
- Keeps JSX cleaner.
- Makes styles easier to organize and reuse.

Recommendation:

For larger applications, prefer organized and reusable styles using StyleSheet or an appropriate styling system.

---

10. What are Components in React Native?

A component is a reusable piece of user interface.

A component can contain:

- Text
- Buttons
- Input fields
- Images
- Other components

Simple Definition:

A component is a function that returns React Native UI elements.

Example

```
import { View, Text } from "react-native";

function Welcome() {
```

```
  return (
    <Text>Welcome Students!</Text>
  );
}

export default function Home() {
  return (
    <View>
      <Welcome />
      <Welcome />
      <Welcome />
    </View>
  );
}
```

Explanation

1. We created a component called Welcome.
2. Welcome returns a Text component.
3. We reused Welcome three times.
4. Each Welcome displays the same message.

Components with Props

Props are used to pass data from a parent component to a child component.

```
import { View, Text } from "react-native";

function Student({ name }) {
  return <Text>Hello {name}</Text>;
}

export default function Home() {
  return (
    <View>
      <Student name="Rahul" />
      <Student name="Priya" />
      <Student name="Amit" />
    </View>
  );
}
```

Output:

```
Hello Rahul
```

```
Hello Priya
Hello Amit
```

Remember:

Components help us reuse UI.

Props help us pass data to components.

---

11. What is Navigation in React Native?

Navigation means moving from one screen to another screen inside a mobile application.

Real-Life Example

Imagine an e-commerce application.

The user opens the Home screen.

Then the user selects a product.

The application opens the Product Details screen.

Then the user opens the Cart screen.

Moving between these screens is called navigation.

Common navigation types include:

1. Stack Navigation
2. Tab Navigation
3. Drawer Navigation

In this lesson, we will focus on Stack and Tab Navigation.

We will use Expo Router, which is commonly included in modern Expo projects.

Expo Router uses files and folders to define application routes.

---

12. Stack Navigation

Stack Navigation is used to move between screens in a stack-like structure.

Real-Life Example

Consider three screens:

Home → Products → Product Details

When the user opens Product Details, the new screen appears above the previous screen.

When the user presses Back, the application can return to Products.

How Stack Navigation Works

Imagine a stack of books.

When you add a book, it goes on top.

When you remove the top book, you see the previous book.

Stack Navigation follows a similar idea.

Step 1: Create the Screens

Inside the app folder, create:

```
app
│
├── _layout.tsx
├── index.tsx
└── details.tsx
```

Step 2: Configure Stack Navigation

File: app/_layout.tsx

```tsx
import { Stack } from "expo-router";

export default function RootLayout() {
  return (
    <Stack>
      <Stack.Screen
        name="index"
        options={{ title: "Home" }}
      />

      <Stack.Screen
        name="details"
        options={{ title: "Details" }}
      />
```

```
    </Stack>
  );
}
```

Explanation:

- Stack is imported from expo-router.
- Stack manages navigation between screens.
- Stack.Screen allows us to configure screen options.
- name identifies the route.
- title defines the header title.

Step 3: Create Home Screen

File: app/index.tsx

```
import { View, Button, Text } from "react-native";
import { router } from "expo-router";

export default function Home() {
  return (
    <View style={{ padding: 20 }}>
      <Text>Welcome to Home Screen</Text>

      <Button
        title="Go to Details"
        onPress={() => router.push("/details")}
      />
    </View>
  );
}
```

Explanation:

router.push() navigates to another route.

Here:

router.push("/details");

Navigates to the Details screen.

Step 4: Create Details Screen

File: app/details.tsx

```
import { View, Text, Button } from "react-native";
```

```
import { router } from "expo-router";

export default function Details() {
  return (
    <View style={{ padding: 20 }}>
      <Text>This is Details Screen</Text>

      <Button
        title="Go Back"
        onPress={() => router.back()}
      />
    </View>
  );
}
```

Explanation:

router.back() returns to the previous screen when there is navigation history.

Important Navigation Methods

router.push()

Pushes a new route onto the navigation stack.

router.push("/details");

router.back()

Returns to the previous screen when possible.

router.back();

router.replace()

Replaces the current route instead of pushing another route onto the stack.

router.replace("/details");

---

13. Tab Navigation

Tab Navigation is used to switch between different main sections of a mobile application.

Usually, tabs are displayed at the bottom of the screen.

Real-Life Example

Consider a shopping application.

At the bottom, we may have:

- Home
- Products
- Profile

When the user presses Home, the Home screen opens.

When the user presses Products, the Products screen opens.

When the user presses Profile, the Profile screen opens.

This is called Tab Navigation.

Step 1: Create Folder Structure

For this separate Tab Navigation example, use:

```
app
│
├── _layout.tsx
│
└── (tabs)
    ├── _layout.tsx
    ├── index.tsx
    ├── products.tsx
    └── profile.tsx
```

Important:

The (tabs) folder is a route group in Expo Router.

Its name does not appear in the URL path.

Step 2: Configure Root Layout

File: app/_layout.tsx

```
import { Stack } from "expo-router";

export default function RootLayout() {
  return (
    <Stack>
      <Stack.Screen
        name="(tabs)"
        options={{ headerShown: false }}
      />
```

```
    </Stack>
  );
}
```

Step 3: Configure Tab Navigation

File: app/(tabs)/_layout.tsx

```
import { Tabs } from "expo-router";

export default function TabLayout() {
  return (
    <Tabs>
      <Tabs.Screen
        name="index"
        options={{ title: "Home" }}
      />

      <Tabs.Screen
        name="products"
        options={{ title: "Products" }}
      />

      <Tabs.Screen
        name="profile"
        options={{ title: "Profile" }}
      />
    </Tabs>
  );
}
```

Explanation:

- Tabs is imported from expo-router.
- Tabs creates tab-based navigation.
- Tabs.Screen configures each tab.
- Each tab connects to a route file.

Step 4: Create Home Screen

File: app/(tabs)/index.tsx

```
import { View, Text } from "react-native";

export default function Home() {
  return (
```

```
    <View style={{ padding: 20 }}>
      <Text>Home Screen</Text>
    </View>
  );
}
```

Step 5: Create Products Screen

File: app/(tabs)/products.tsx

```
import { View, Text } from "react-native";

export default function Products() {
  return (
    <View style={{ padding: 20 }}>
      <Text>Products Screen</Text>
    </View>
  );
}
```

Step 6: Create Profile Screen

File: app/(tabs)/profile.tsx

```
import { View, Text } from "react-native";

export default function Profile() {
  return (
    <View style={{ padding: 20 }}>
      <Text>Profile Screen</Text>
    </View>
  );
}
```

Output

The application displays three tabs:

Home | Products | Profile

When the user selects a tab, its corresponding screen is displayed.

---

14. Stack Navigation vs Tab Navigation

Stack Navigation

Purpose:

Move forward and backward between screens.

Example:

Home → Products → Product Details

Common Use Cases:

- Login to Dashboard
- Product List to Product Details
- Profile to Edit Profile

Tab Navigation

Purpose:

Switch between the main sections of an application.

Example:

Home | Products | Profile

Common Use Cases:

- Shopping applications
- Social media applications
- Banking applications

Can We Use Both Together?

Yes.

In real-world applications, Stack and Tab Navigation are often used together.

For example:

An application may have three bottom tabs:

Home | Products | Profile

Inside the Products section, we may navigate from the Products screen to the Product Details screen using Stack Navigation.